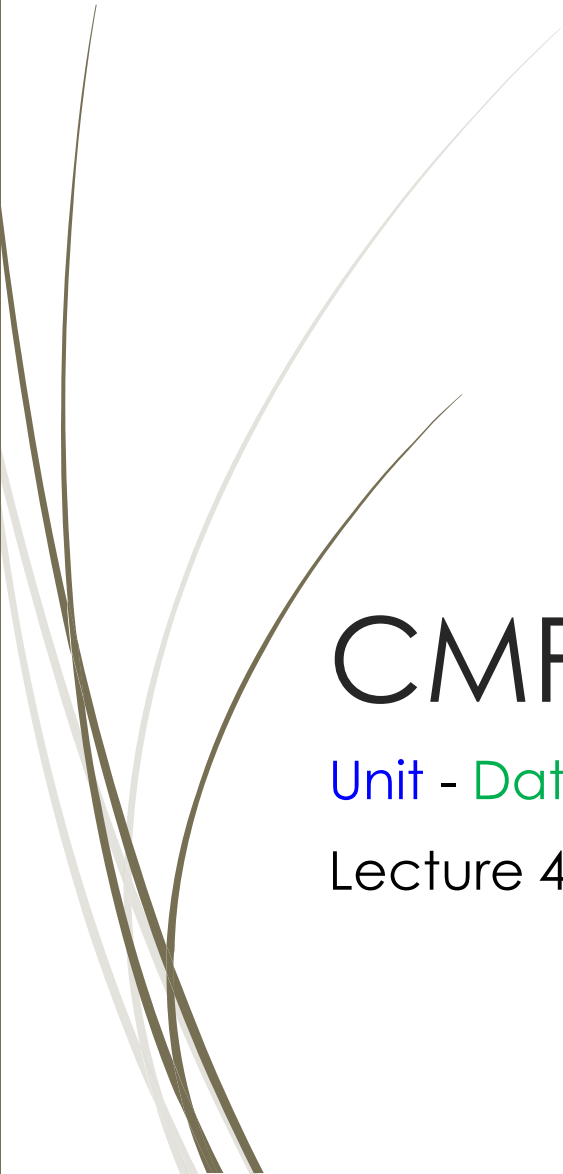




CMPT 295

Unit - Data Representation

Lecture 4 – Representing (encoding) integral numbers in memory
– Arithmetic operations

Last Lecture

reviewed in Lab 1

Seen in the demo

We shall see
the demo
today!

- Interpretation (encoding) of bit pattern **B** into either unsigned value **U** or signed value **T**
 - **B2U(X)** and **U2B(X)** encoding schemes (conversion)
 - **B2T(X)** and **T2B(X)** encoding schemes (conversion)
 - Signed value expressed as two's complement => **T**
- Conversions from unsigned <=> signed values
 - Method 1: **U2T(X)** and **T2U(X)** => **U2B(X)** then **B2T(X)**, **T2B(X)** then **B2U(X)**
 - Method 2: **U2T(X)** and **T2U(X)** => adding or subtracting 2^w
- Implication in **C**: when converting (implicitly via promotion and explicitly via casting):
 - Sign:
 - Unsigned <=> signed (of same size) -> **NOTE**: Both have same bit pattern, however, this bit pattern may be interpreted differently
 - Can have unexpected effects, i.e., producing a different value
 - Size:
 - Small -> large (for signed, e.g., **short** to **int** and for unsigned, e.g., **unsigned short** to **unsigned int**)
 - **sign extension**: For unsigned -> zeros extension, for signed -> sign bit extension
 - Both yield expected result -> **resulting value is unchanged**
 - Large -> small (e.g., **unsigned int** to **unsigned short**)
 - **truncation**: Unsigned/signed -> most significant bits are truncated (discarded)
 - May not yield expected results -> **original value may be altered**
 - Both (sign and size): 1) **size** conversion is executed first, then 2) **sign** conversion is executed
e.g., **int** to **unsigned short**: **int** to **unsigned int** then to **unsigned short**

Today's Menu

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation

- Floating point in C – casting, rounding, addition, ...

Warm up question

- ▶ What is the value of ...
 - ▶ TMin (in hex) for `signed char` in C: _____
 - ▶ TMax (in hex) for `signed int` in C: _____
 - ▶ TMin (in hex) for `signed short` in C: _____

Integer addition (unlimited space – on paper)

- What happens when we add two decimal numbers?

$$\begin{array}{r} 1 \text{ } \leftarrow \text{carry in} \\ 107_{10} \\ + 938_{10} \\ \hline \text{carry out } \rightarrow 1045_{10} \text{ True sum} \end{array}$$

- Same thing happens when we add two binary numbers:

$$\begin{array}{r} 11 \text{ } \leftarrow \text{carry in} \\ 101100_2 \\ + 101110_2 \\ \hline \text{carry out } \rightarrow 1011010_2 \text{ True sum} \end{array}$$

Unsigned addition (limited space - in memory) => memory has a fixed size

► What happens when we add two unsigned values:

w = 8

a)

$$\begin{array}{r} \overset{1\ 1\ 1\ 1\ 1}{00111011}_2 \quad \leftarrow \text{carry in} \\ + 01011010_2 \\ \hline 10010101_2 \end{array}$$

59_{10}
 $+ 90_{10}$
 149_{10}

Actual sum

True sum

$$\begin{aligned} &10010101_2 \\ &= 128 + 16 + 4 + 1 \\ &= 149_{10} \end{aligned}$$

b)

$$\begin{array}{r} \overset{\leftarrow \text{carry in}}{10101110}_2 \quad \leftarrow \text{carry in} \\ + 11001011_2 \\ \hline \end{array}$$

174_{10}
 $+ 203_{10}$
 377_{10}

Actual sum

True sum

carry out ->

$w = 8 \rightarrow [0..255]$

Closer look at unsigned addition overflow

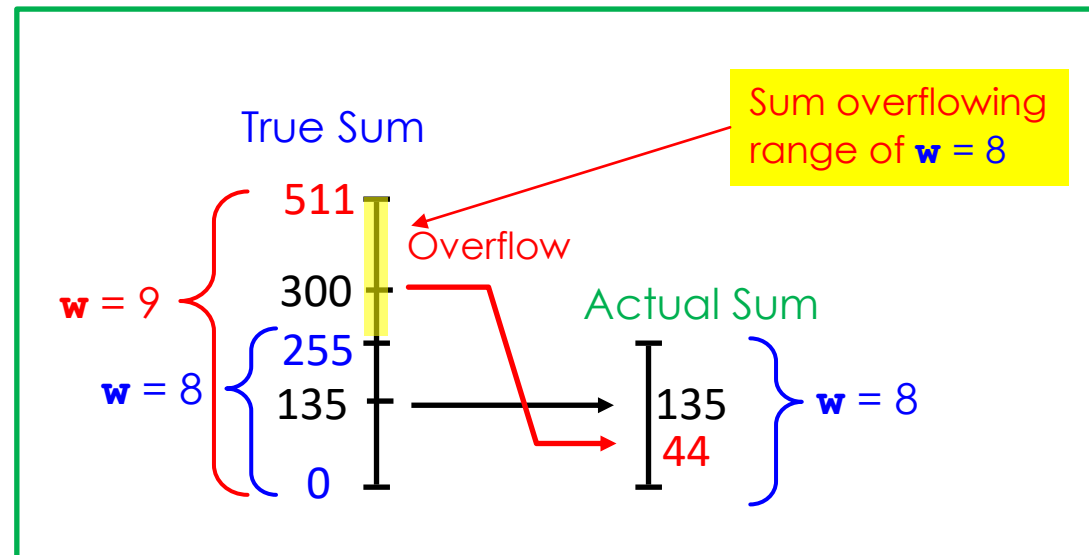
a)

	carry in	1111
90_{10}		01011010_2
$+ 45_{10}$		$+ 00101101_2$
True sum		Actual sum
135_{10}		10000111_2
carry out		

b)

	carry in	11111111
255_{10}		11111111_2
$+ 45_{10}$		$+ 00101101_2$
True sum		Actual sum
300_{10}		100101100_2
carry out		

$255_{10} = 11111111_2$
 $135_{10} = 10000111_2$
 $90_{10} = 01011010_2$
 $45_{10} = 00101101_2$
 $44_{10} = 00101100_2$



Unsigned addition ($+^u_w$) and overflow

Result (i.e., expected sum) of **integer addition** with unlimited space

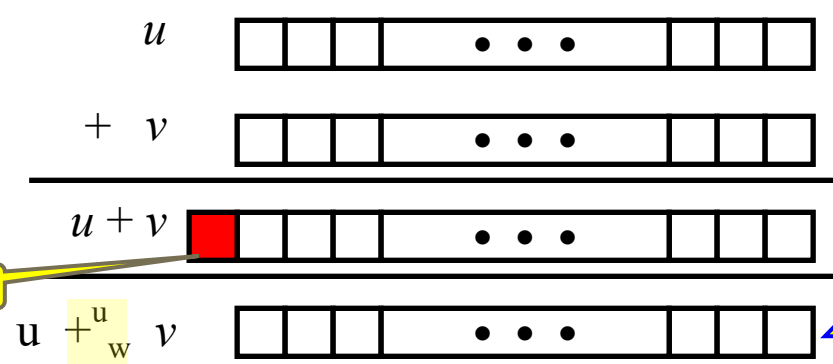
Operands: w bits

True Sum: $w + 1$ bits

This bit overflows the memory space

Actual Sum (w bits):

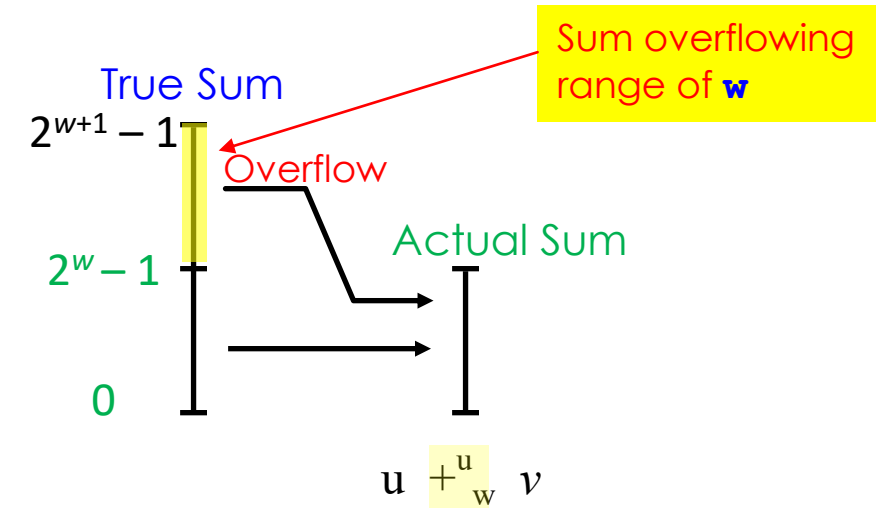
Discard **carry out bit**



Result of **unsigned addition** with limited space

- Discarding **carry out bit** has same effect as applying modular arithmetic

$$s = u +^u_w v = (u + v) \bmod 2^w$$

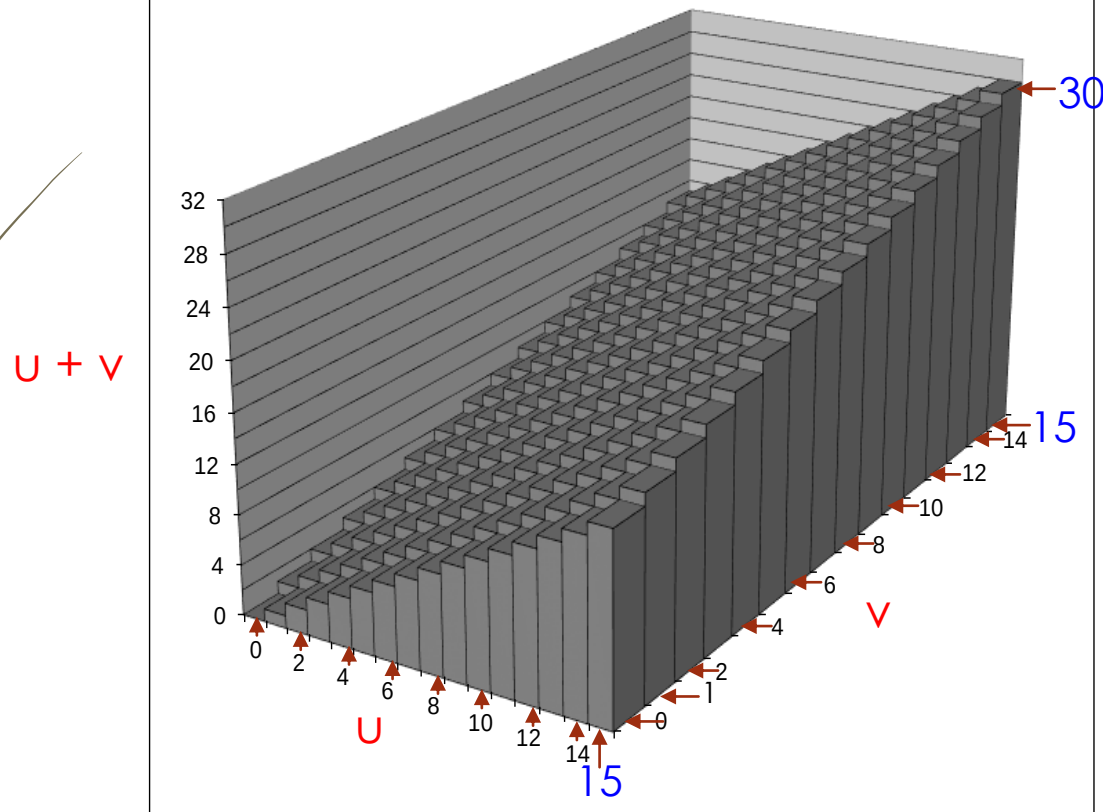


Comparing integer addition with unsigned addition ($w = 4$)

Overflow: Effect of fixed-size memory

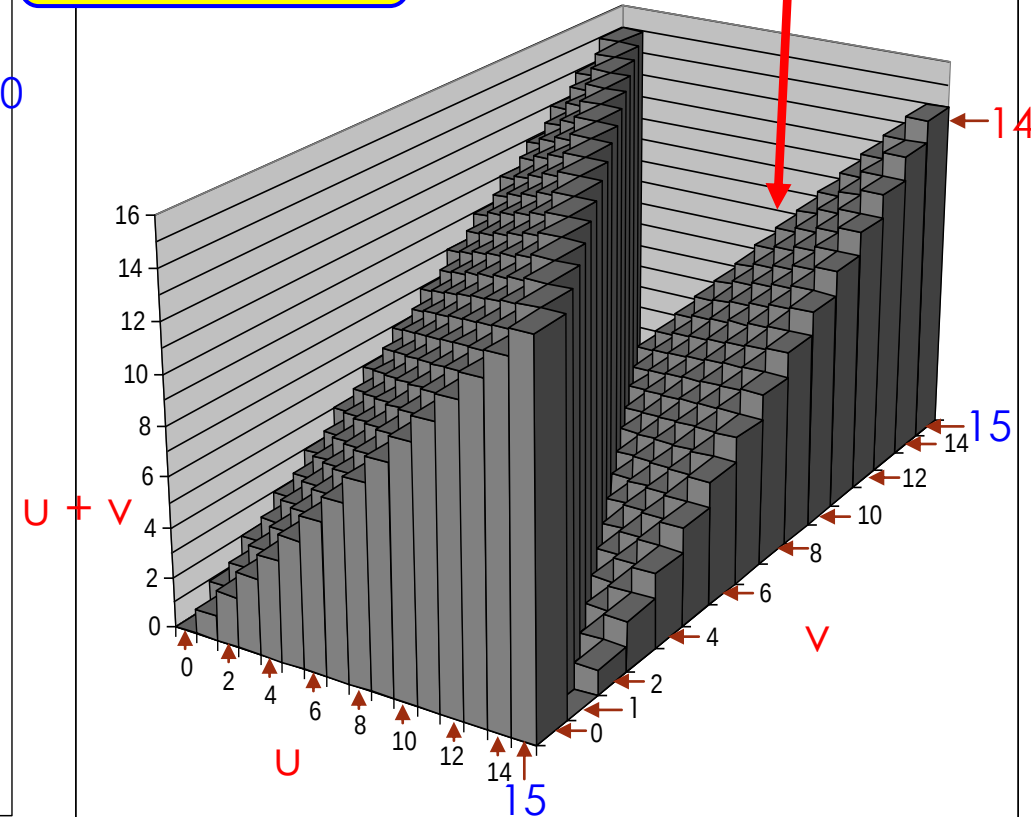
with unlimited space

Integer Addition



with limited space (fixed size memory)

Unsigned Integer Addition



An overflow occurs when there is a **carry out**

For example: $15 (1111_2) + 15 (1111_2) = 30 (11110_2 \leftarrow \text{true sum})$ and $= 14 (\cancel{1}1110_2 \leftarrow \text{actual sum})$

Signed addition (limited space - in memory) => memory has a fixed size

➤ What happens when we add two signed values:

$w = 8$

a)
$$\begin{array}{r} \text{1 1 1 1 1} \text{ <- carry in} \\ 00111011_2 \\ + 01011010_2 \\ \hline 10010101_2 \end{array}$$

Actual sum

$$\begin{array}{r} 59_{10} \\ + 90_{10} \\ \hline 149_{10} \end{array}$$

True sum

$$\begin{array}{l} 10010101_2 \\ = -128 + 16 + 4 + 1 \\ = -107_{10} \end{array}$$

b)
$$\begin{array}{r} \text{<- carry in} \\ 10101110_2 \\ + 11001011_2 \\ \hline \end{array}$$

Actual sum

$$\begin{array}{r} -82_{10} \\ + -53_{10} \\ \hline -135_{10} \end{array}$$

True sum

Observation: Unsigned and signed additions have identical behaviour @ the bit level, i.e., their sum have the same bit pattern, but the interpretation of this bit pattern differs

$w = 8 \rightarrow [-128..127]$

Closer look at signed addition overflow

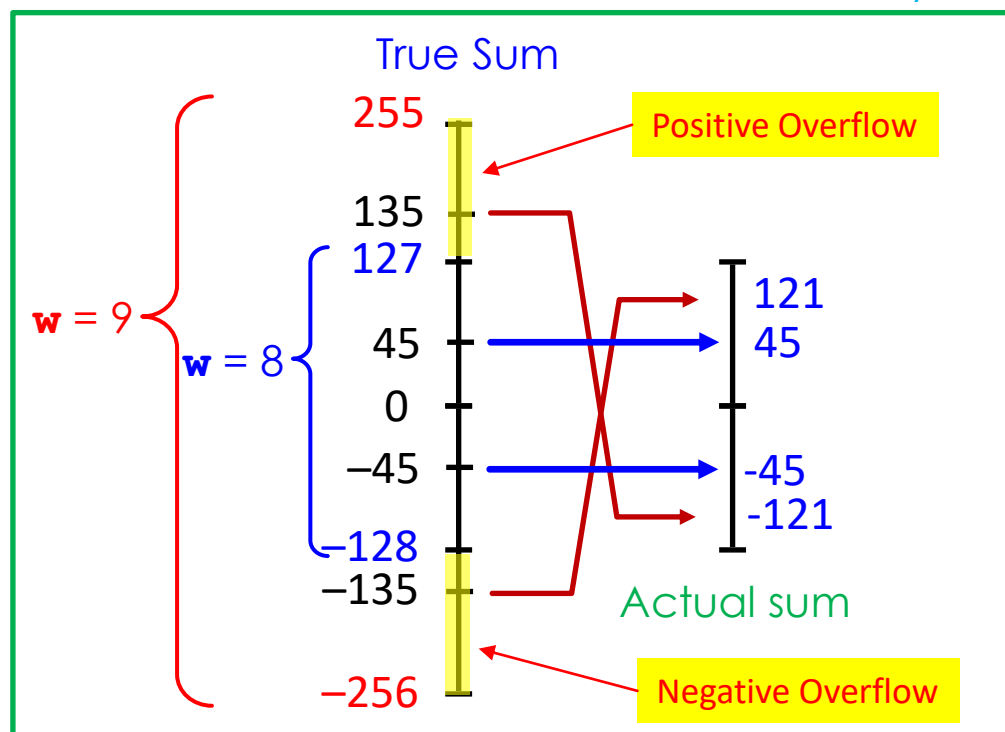
a)

	carry in	1 1 1 1	
90_{10}		01011010_2	
$+ 45_{10}$		$+ 00101101_2$	
<u>True sum</u>		<u>10000111_2</u>	Actual sum
135_{10}			≤ -121
	carry out		

b)

	carry in	1 1	
-90_{10}		10100110_2	
$+ -45_{10}$		$+ 11010011_2$	
<u>True sum</u>		<u>101111001_2</u>	Actual sum
-135_{10}			≤ 121
	carry out		

$121_{10} = 01111001_2$
 $90_{10} = 01011010_2$
 $45_{10} = 00101101_2$
 $-45_{10} = 11010011_2$
 $-90_{10} = 10100110_2$
 $-121_{10} = 10000111_2$



Closer look at signed addition overflow

c)

carry in	1	1	1
-90 ₁₀	10100110 ₂		
+ 45 ₁₀	+ 00101101 ₂		
True sum	-45 ₁₀	Actual sum	11010011 ₂
carry out			

d)

carry in	1	1	1
90 ₁₀	01011010 ₂		
+ -45 ₁₀	+ 11010011 ₂		
True sum	45 ₁₀	Actual sum	100101101 ₂
carry out			

Signed addition ($+^t_w$) and overflow

Result (i.e., expected sum) of **integer addition** with unlimited space

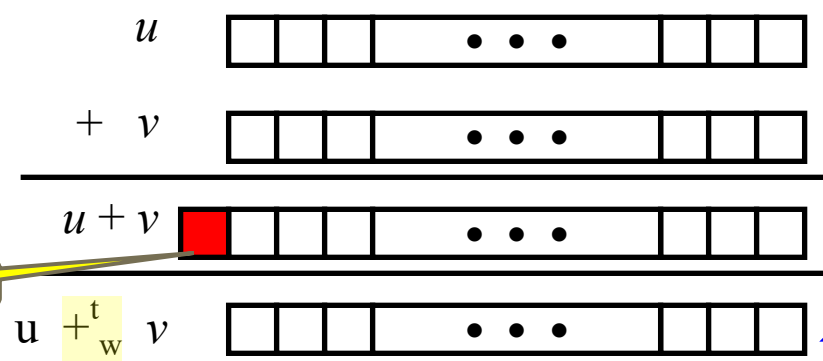
Operands: w bits

True Sum: $w + 1$ bits

This bit overflows the memory space

Actual Sum (w bits):

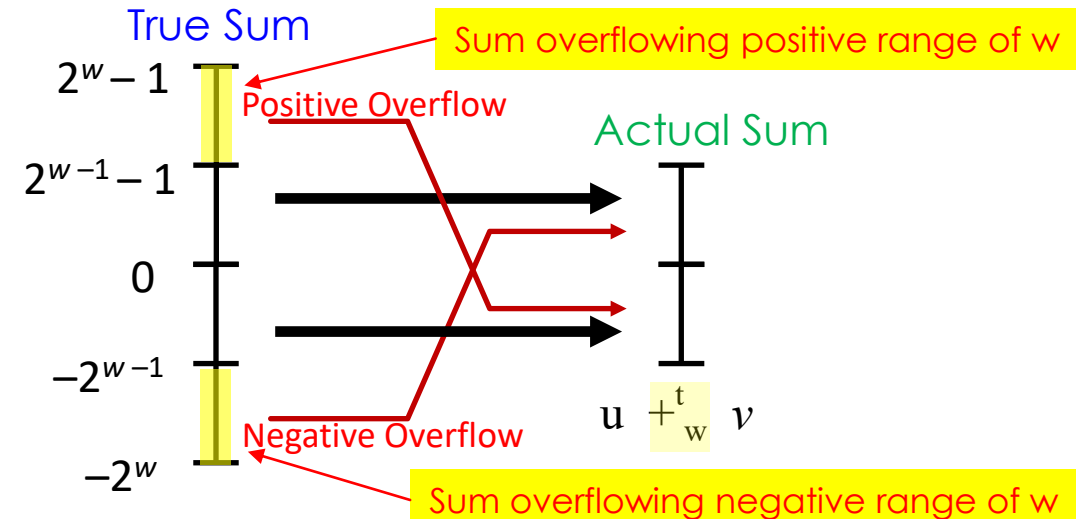
Discard carry out bit



Result of **signed addition** with limited space

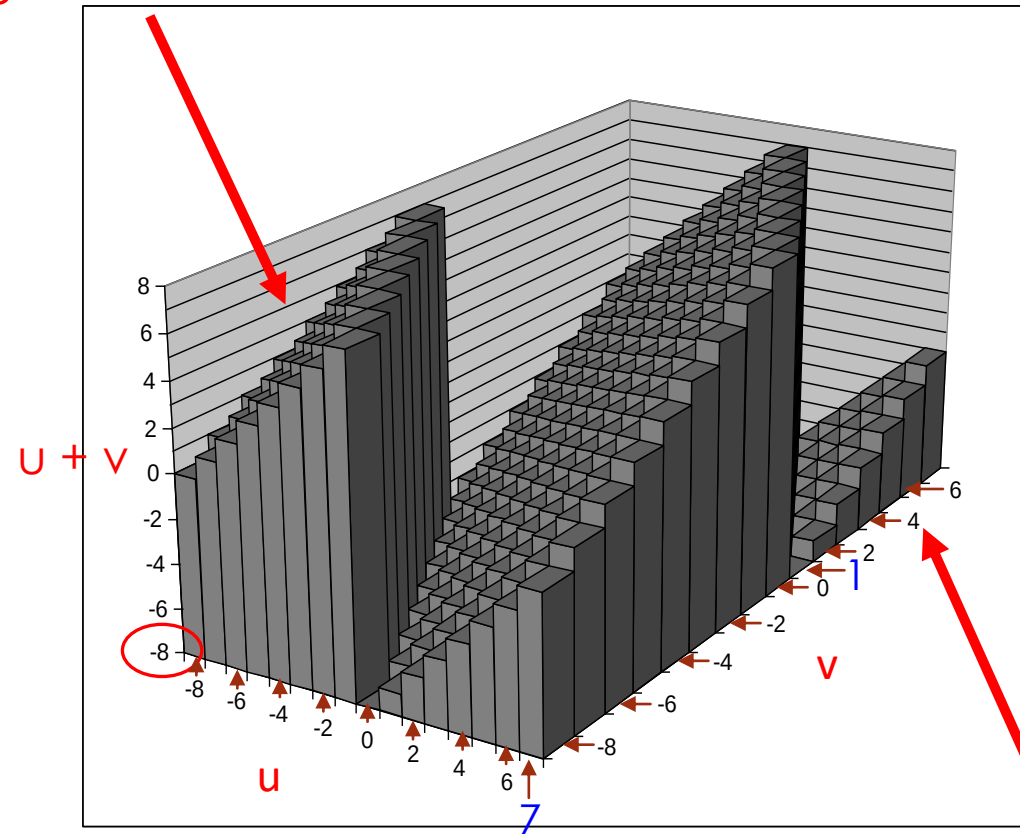
- Discarding carry out bit has same effect as applying modular arithmetic

$$s = u +^t_w v = U2T_w [(u + v) \bmod 2^w]$$



Visualizing signed addition overflow ($w = 4$)

Negative Overflow



Positive Overflow

For example: $7 (0111_2) + 1 (0001_2) = 8 (1000_2 \leftarrow \text{true sum})$ and $= -8 (1000_2 \leftarrow \text{actual sum})$

What about subtraction? -> Addition

$$x + (-x) = 0$$

- Subtracting a number is equivalent to adding its additive inverse
 - Instead of subtracting a positive number, we could add its negative version:

$$\begin{array}{r} 107 \\ -118 \\ \hline -11 \end{array} \Rightarrow \begin{array}{r} 107 \\ +(-118) \\ \hline ? \end{array}$$

- Let 's try: $107_{10} \rightarrow 01101011_2 \rightarrow 01101011_2$

$$\begin{array}{r} -118_{10} \\ \hline -11 \end{array} \rightarrow \begin{array}{r} -01110110_2 \\ \hline \end{array} \rightarrow + \begin{array}{r} 10001010_2 \\ \hline \end{array}$$

$$-11$$

$$11110101_2 \Rightarrow -11_{10}$$

T2B(X) conversion: $(\sim(\mathbf{U2B}(|X|)))+1$
= $(\sim(\mathbf{U2B}(|-118|)))+1$
= $(\sim(\mathbf{U2B}(118)))+1$
= $(\sim(01110110_2))+1$
= $(10001001_2)+1$
= 10001010_2

$$\text{CHECK: } -128+64+32+16+4+1 = -11_{10}$$

Multiplication ($*^u_w, *^t_w$) and overflow

Would be the result (i.e., expected product) of **integer multiplication** with unlimited space

Operands: w bits

At most

These bits overflow the memory space

True Product: $2w$ bits

Actual Product (w bits):

Discard MS (i.e., high order) w bits

- Discarding high order w bits has same effect as applying modular arithmetic

$$p = u *^u_w v = (u * v) \bmod 2^w$$

$$p = u *^t_w v = U2T_w [(u * v) \bmod 2^w]$$

- Example: $w = 4$

5_{10}	0101_2
$\times 5_{10}$	$\times 0101_2$
25_{10}	0101_2
	0000_2
	0101_2
	0000_2
	0011001_2

Result of **multiplication** with limited space

Two's Complement Multiplication

By Karen Miller, 2006

Here are a couple of ways of doing two's complement multiplication by hand. Direct implementations of these algorithms into the circuitry would result in very slow multiplication! Actual implementations are far more complex, and use algorithms that generate more than one bit of product each clock cycle. ECE 352 should cover these faster algorithms and implementations.

Remember that the result can require 2 times as many bits as the original operands. We can assume that both operands contain the same number of bits.

"No Thinking Method" for Two's Complement Multiplication

In 2's complement, to always get the right answer without thinking about the problem, *sign extend* both integers to twice as many bits. Then take the correct number of result bits from the least significant portion of the result.

A 4-bit, 2's complement example:

1111 1111	-1
x 1111 1001	x -7
-----	-----
11111111	7
00000000	
00000000	
11111111	
11111111	
11111111	
11111111	
+ 11111111	

1 0000000111	

	(correct answer underlined)

Another 4-bit, 2's complement example, showing both the incorrect result (when sign extension is not done), and the correct result (with sign extension):

WRONG ! <pre> 0011 (3) x 1011 (-5) ----- 0011 0011 0000 + 0011 ----- 010001 not -15 in any representation! </pre>	Sign extended: <pre> 0000 0011 (3) x 1111 1011 (-5) ----- 00000011 00000011 00000000 00000011 00000011 00000011 00000011 + 00000011 ----- 101111001 ----- take the least significant 8 bits 11110001 = -15 </pre>
--	--

"Slightly Less Work Method" for Two's Complement Multiplication

```

multiplicand
x multiplier
-----
product

```

If we do *not* sign extend the operands (multiplier and multiplicand), before doing the multiplication, then the wrong answer sometimes results. To make this work, *sign extend the partial products* to the correct number of bits.

To result in the least amount of work, classify which do work, and which do not.

+	-	+	-	(muliplicands)
x +	x +	x -	x -	(mulipiers)
----	----	----	----	
OK	sign extend partial products	 take additive inverses		
		-	+	
		x +	x +	
		----	----	
	sign extend partial products		OK	

Example:

without
sign extension

```

11100 (-4)
x 00011 (3)
-----
11100
11100
-----
1010100 (-36)
WRONG!

```

with correct
sign extension

```

11100
x 00011
-----
111111100
111111100
-----
1111110100 (-12)
RIGHT!

```

Another example:

without adjustment

```

11101 (-3)
x 11101 (-3)
-----
11101
11101
11101
+11101
-----
1101001001 (wrong!)

```

with correct adjustment

```

11101 (-3)
x 11101 (-3)
-----
      (get additive inverse of both)
00011 (+3)
x 00011 (+3)
-----
00011
+ 00011
-----
001001 (+9) (right!)

```

Multiplication with power-of-2 versus shifting

- If $x * y$ where $y = 2^k$ then $x * y \Rightarrow x \ll k$
 - For both signed and unsigned
- Example:
 - $x * 8 = x * 2^3 \rightarrow x \ll 3$
 - $x * 24 = (x * 2^5) - (x * 2^3) = (x * 32) - (x * 8) \rightarrow (x \ll 5) - (x \ll 3)$
(decompose 24 in powers of 2 $\Rightarrow 32 - 8$)
- Most machines shift and add faster than multiply
 - We'll soon see that compiler generates this code automatically

Summary

Addition:

Unsigned/signed:

- Behave the same way at the bit level
- Interpretation of resulting bit pattern (sum) may differ

- Unsigned addition $\rightarrow$ true sum may overflow its w bits in memory
 - If so, then actual sum $= (x + y) \bmod 2^w$ (equivalent to subtracting 2^w from true sum $(x + y)$)

- Signed addition $\rightarrow$ true sum may overflow its w bits in memory

If so then ...

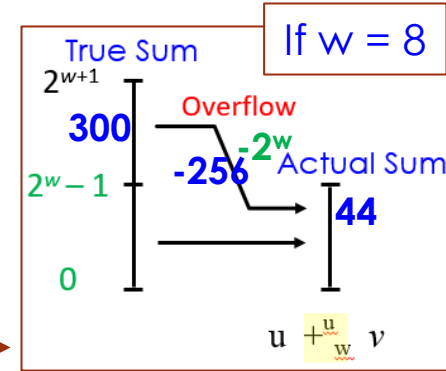
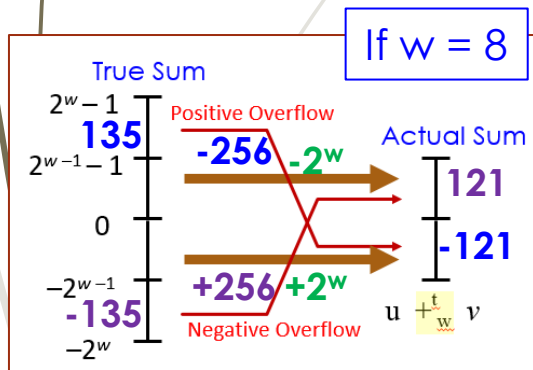
- actual sum $= \text{U2T}_w [(x + y) \bmod 2^w]$
 - $90 + 45 = 135 \bmod 256 \Rightarrow 135 - 256 = -121$
 - $-90 + -45 = -135 \bmod 256 \Rightarrow -135 + 256 = 121$
- true sum may be too +ve $\rightarrow$ positive overflow OR too -ve $\rightarrow$ negative overflow
 - $90 + 45 = 135$ but get -121
 - $-90 + -45 = -135$ but get 121

Subtraction

- Becomes an addition where the 2nd operand is transformed into its additive inverse in two's complement

- Multiplication: true product may overflow its w bits in memory, if so then...

- Unsigned: actual product $= (x * y) \bmod 2^w$
- Signed: actual product $= \text{U2T}_w [(x * y) \bmod 2^w]$
- Can be replaced by additions and shifts



Next lecture

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

Demo

- Bits representing (encoding) real numbers

- IEEE floating point representation

- Floating point in C – casting, rounding, addition, ...